

无线电干扰源的快速自动定位与清除模型

摘要

【待填充：开头总述，一句话交代背景与要解决的问题，一句话说明本文总体上采用什么模型与算法体系，并串起四问“单次定位 → 检测点布置 → 多源清除 → 定向源辨识”。】

针对问题一，基于示向度误差有界的特点，本文建立了有界区间角楔交会模型，把每个检测点的示向度按误差界张成角楔，干扰源必在各角楔交会的公共部分中。由于形成的定位区域为凸多边形，直径归结为顶点集上的最远点对。使用程序模拟构造的五组算例直径在 10.59 ~ 25.64 m 之间，而以该直径为直径的圆并不总能覆盖定位区域，五组算例中有两组失败，越界顶点到圆心距离超过半径约 10.6%。

针对问题二，只掌握一个检测点处的示向度、源点距离未知，无法先定位再布点，故建立以期望定位区域直径为准则的布点模型：由两探测束的交会几何写出定位区域直径，并对源点位置与第二次测向的误差求取期望，得到只依赖第二个检测点相对第一个检测点位置 (a, b) 的二维价值函数 $E[D](a, b)$ ，使用理论方法求解与数值模拟方法求解获得结果吻合。我们选择一个合理阈值 60m，则第二检测点和候选区域即为相对位置满足 $E[D] < 60m$ 的区域 $C = \{(a, b) | E[D](a, b) < 60\}$ ，具体区域形状见下正文。

针对问题三，【待填充：多源定位与清除策略及实测指标。】

针对问题四，【待填充：含定向源的检测与清除策略及实测结果。】

【待填充：结尾总评，四个模型的优点、缺点各一两条，并给出可继续改进的方向。】

关键词：示向度交会 直径圆覆盖

一、问题背景与重述

1.1 问题背景

随着无线电技术的普及和广泛应用，频谱管理中的干扰源定位与清除问题日益突出。干扰不仅影响正常通信，还可能危及公共安全。传统清除方式通常受环境遮挡、低功率发射等影响，精准定位耗时长，且危险环境下人工作业风险高，因此研发机器狗自动定位清除系统代替人工执行任务具有现实需求。但任务中通常面临着目标区域范围较大，干扰源数量、频道、位置、有效接收半径及类型未知，测向存在误差，移动、检测、切换频道和清除过程存在耗时，定向干扰源的发射方向无法辨识等困难。如何在信息不完全且时间受限的条件下高效完成任务，是该自动定位清除系统亟待解决的问题。

1.2 问题重述

考虑到上述实际情况中面临的困难，本文需依次解决以下四个问题：

1. 已知若干检测点的位置及干扰源关于这些检测点的示向度，给出用交会定位法计算可能位置区域（多边形）直径的算法，并回答以该直径为直径的圆能否覆盖此区域。
2. 已知在某一个检测点处测得的示向度，给出第二个检测点的选择策略，并指出该点可取在什么范围（候选区域）之内。
3. 区域内存在多个干扰源，它们占用互不相同的频道且总个数未知，机器狗从给定起点出发自动定位并逐个清除。要求给出策略与算法，经模拟器演练检验后给出清除比例与平均定位清除时间，并据以完成正式测试。
4. 区域内除全向干扰源外还混有定向干扰源，其个数与发射方向均未知，其余条件与问题三相同。要求给出相应的检测与清除策略及算法。

二、问题分析

四个子问题层层递进。问题一是单次交会定位的几何基础；问题二则将该几何模型进一步用作检测点位置的选择依据，形成单源定位策略；问题三在严格的时间预算下，把单源定位能力嵌入“测向、收敛、清除”的在线决策；问题四基于问题三模型的基础之上，再额外引入定向源方向诊断对决策的影响。

2.1 问题一的分析

本问不宜假设误差分布，而应利用题目给出的误差界，采用最坏情形模型：每个检测点的示向度对应一个由“测量方向 $\pm 1^\circ$ ”张成的角楔。多个检测点的角楔求交，即得干扰源的可能定位区域。由于角楔是凸集，其交仍为凸集（有界时即为凸多边形），因此区域直径可转化为凸多边形顶点间的最远点对问题。求解时，需要先判断交会区域是否有界，再检验以该直径为直径的圆能否覆盖定位区域。整体求解思路可概括为：由误差界构造单点角楔，通过多检测点角楔交会确定凸多边形定位区域，进而将区域直径计算归结为顶点间的最远点对问题；同时辅以有界性判定与圆覆盖检验。

2.2 问题二的分析

问题的关键在于第二个检测点无法由单次观测唯一确定：只掌握 S_1 处的示向度，源点的距离信息完全缺失，故既不能先定位再布点，也不存在一个位置在每次试验中都最好。本文改以“布点后的定位效果之期望”反向衡量布点位置，沿用问题一中“定位区域直径”这一判据，取其对第二次检测点示向度 θ_2 的测量不确定性与源点位置的不确定性的期望为关键判据，既不依赖某一次定位的运气，也无需先知道源点的距离。又因直径的期望只由两次探测的交会几何决定，它只依赖第二个检测点相对第一个检测点的位置，与绝对坐标无关。于是本问归结为以第二检测点对第一检测点的相对位置为决策变量、以 $E[D](a, b)$ 为价值函数的二维寻优问题，其等值线内部有界域即第二个检测点的候选区域。

2.3 问题三的分析

【待填充：给出策略与算法，并补充模拟与实测指标。】

2.4 问题四的分析

【待填充：给出定向源的检测与清除策略、诊断规则及实测结果。】

三、模型假设

为便于建模求解，在合理范围内作如下假设：

1. 示向度误差有界， $|\Delta\theta_i| \leq \varepsilon$ （本题 $\varepsilon = 1^\circ$ ），且同一检测点的误差在观测期间固定不变，只有更换地点才呈现统计规律；故采用有界区间模型（最坏情形），所得区域保证包含真值。
2. 检测点坐标与示向度读数精确已知；示向度以 x 轴正向逆时针为正，取值于 $[0^\circ, 360^\circ)$ 。
3. 干扰源位置在观测与清除期间固定不变，各源频道互不相同且不随时间变化。
4. 测向机与干扰源位于同一平面，区域内无障碍遮挡。
5. 全向源的覆盖角度为 360° ，定向源为定向方向两侧各 90° （含边界），覆盖范围内信号均匀辐射、场强仅随距离衰减。
6. 机器狗沿直线以 5 m/s 匀速行进，忽略转向与加减速耗时；移动中不可检测，单次检测 5 s ，任意两频道切换 1 s 。
7. 与干扰源距离不超过 20 m 时，光学定位（ 3 s ）与激光清除（ 2 s ）必定成功且与覆盖角度无关；距离不超过 5 m 且处于覆盖角度内时无法获得示向度，可直接定位并清除。

四、符号说明

本文使用的主要符号见表 1，未列出的符号在首次出现处说明。

表 1 主要符号说明

符号	含义	单位
$S_i = (x_i, y_i), \theta_i$	第 i 个检测点的坐标与示向度	m, °
$T = (x_T, y_T)$	干扰源（待定位目标）的真实位置	m
$\Delta(\alpha, \beta)$	两方位角之差，归一到 $(-180^\circ, 180^\circ]$	°
ε	示向度误差界（本题 1° ）	°
$\mathcal{R}, V$	定位区域及其顶点集	无
D	定位区域直径	m
M	直径圆圆心（直径中点）	m
A, B	直径端点	m
$S_1, S_2 = (a, b)$	第一、第二个检测点及其相对位置	m
θ_2	第二个检测点处的示向度	°
φ	候选区域极角（自 S_1 的示向度方向向量起）	°
x_{max}	干扰源最大可被有效接收距离	m
$E[D](a, b)$	定位区域直径的期望（第二个检测点的价值函数）	m

五、模型建立与求解

5.1 问题一：示向度交会定位模型

5.1.1 模型建立

设第 i 个检测点为 $S_i = (x_i, y_i)$ ，测得的示向度为 θ_i （ x 轴正向逆时针，取值 $[0^\circ, 360^\circ)$ ），误差界为 $\varepsilon = 1^\circ$ 。干扰源则落在以 S_i 为顶点、关于示向度方向对称张开 2ε 的角楔之内（为避免方位角相减在跨 0° 时出错，记两角之差 $\Delta(\alpha, \beta) = ((\alpha - \beta + 180^\circ) \bmod 360^\circ) - 180^\circ$ ）：

$$\mathcal{W}_i = \{S_i\} \cup \{T \neq S_i : |\Delta(\beta(S_i, T), \theta_i)| \leq \varepsilon\}, \quad (1)$$

其中 $\beta(S, T)$ 为自检测点 S 指向干扰源 T 的方位角。 m 次测向给出的定位区域即各角楔之交，如图 1 所示（以 $m = 3$ 为例）。每个角楔可表示为两个闭半平面之交；各角楔对干扰源的约束同时成立，故定位区域为 $\mathcal{R} = \bigcap_{i=1}^m \mathcal{W}_i$ ，它是 $2m$ 个半平面之交，为凸多边形区域，可能为空、无界，或退化为点、线段。非空有界时其顶点数不超过 $2m$ ，且每个顶点必是某两条边界直线的交点。记该区域的顶点集为 V 。

若各示向度方向过于接近， $\mathcal{R}$ 沿公共方向无限延伸，非空定位区域的直径为 $+\infty$ ，故在确认非空后须判定有界性。由于无界是无穷远处的性质，判据只涉及方向：角楔是顶点在 S_i 的锥，若某方向同时落在各角楔张开的方向区间内，则区域内任一点沿该方向平移任意远都不会离开区域，区域因而无界；反之，若区域有界，取其中趋于无穷的一列点单位化，由单位圆的紧性可取收敛子列，而自各 S_i 指向该点列的方位角都趋于该极限方向，角楔又是闭集，故极限方向必落在每个区间内。于是对于非空 $\mathcal{R}$ ，无界当且仅当各方位角区间 $\Theta_i = [\theta_i - \varepsilon, \theta_i + \varepsilon]$ （跨 0° 按圆周区间处理）存在公共交集，当 $m = 2$ 时即两次测向的方向夹角不超过 2ε 。该判据只需 $O(m)$ 次角度比较，且不依赖检测点坐标。

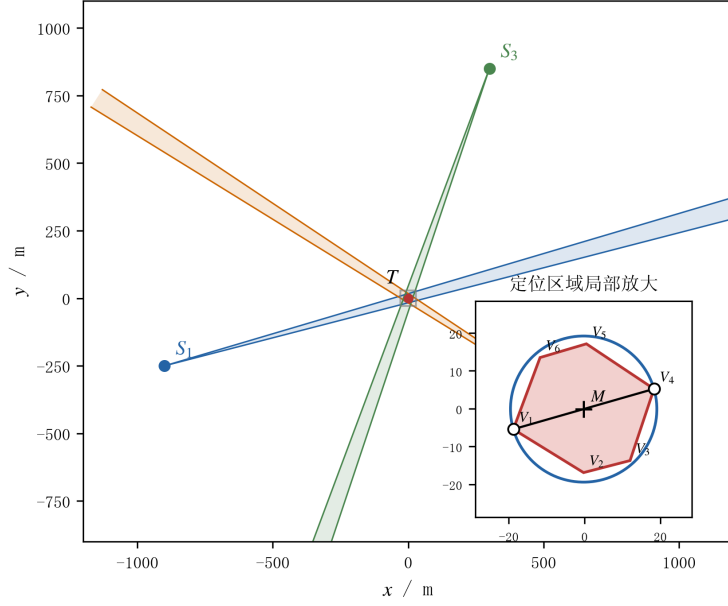


图 1 多检测点示向度交会得到的定位区域；右下角的局部放大图中标出交点 V_i 和直径中点 M

对已判定为有界的区域，凸多边形上任意两点的最大距离必由两个顶点达到。故求直径不必在连续区域上作最优化，只需在顶点集 V 上比较点对距离：

$$D = \max_{P, Q \in \mathcal{R}} \|P - Q\| = \max_{P, Q \in V} \|P - Q\|. \quad (2)$$

设使式 (2) 取到最大值的两个顶点为 A, B ，则直径圆圆心为 $M = (A + B)/2$ 、半径为 $r = D/2$ 。

覆盖与否由圆盘的凸性直接判定： $\mathcal{R} \subseteq$ 该直径圆当且仅当其全部顶点含于圆内，即

$$\max_{P \in V} \|P - M\| \leq D/2. \quad (3)$$

条件不满足时，必有顶点 P 使 $\angle APB < 90^\circ$ 而落在圆外。注意该直径圆的圆心固定为 M ，与圆心可自由选取的最小包围圆不同，故式 (3) 需逐个顶点检验。注意到最远点对可能不唯一，但覆盖判定与取法无关：若某一对最远点 A, B 确定的圆已覆盖 $\mathcal{R}$ ，则 $\mathcal{R}$ 含于半径 $D/2$ 的圆内，而 $\mathcal{R}$ 内任意两点距离不超过 D ，故任何其他最远点对都必对应于该圆上的一条直径，其确定的圆与原圆重合；若某一对不能覆盖，则任何一对也都不能。因此下文任取一对最远点构造直径圆即可。

5.1.2 模型求解算法

算法输入为 m 组“检测点坐标与示向度”及误差界 ε ，输出为顶点集 V 、直径 D 、直径端点 A, B 、圆心 M 及覆盖判定，流程如图 2，分四步。

第一步（区域构造与非空检验）：区域的每个顶点都是两条边界直线的交点，故算法枚举每个角楔两条边界射线所在直线（共 $2m$ 条）的两两交点，再用全部 $2m$ 个半平面判据逐一筛选；同一角楔两条边界线的交点即检测点 S_i ，它同样作为候选点保留。随后对候选点集求凸包（包含全部候选点的最小凸多边形，用 Andrew 单调链实现），剔除重复点与近似共线点，得到逆时针排列的顶点表 V 。若候选点集为空，说明各角楔无公

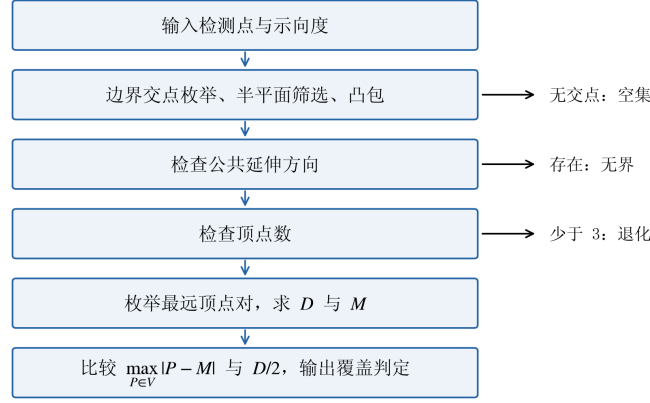


图 2 问题一求解算法流程

共交集（测量数据自相矛盾），算法返回状态 `empty`（无可行定位区域）并终止该例计算。

第二步（有界性与退化检验）：对非空交集，检查 m 个方位角区间 $[\theta_i - \varepsilon, \theta_i + \varepsilon]$ 是否存在公共方向。若存在，返回 `unbounded`；若不存在且 $|V| < 3$ ，返回 `degenerate`，表示点或线段。其余情形得到有界凸多边形，继续计算直径。

第三步（求直径）：按式 (2) 枚举 V 中的顶点对取最大距离，对应两点记为 A, B ，圆心取 $M = (A + B)/2$ ；对本文算例规模直接枚举即可，计算量为 $O(|V|^2)$ 。程序在顶点较多时使用旋转卡壳算法，其距离比较次数随顶点数线性增长；另以两法交叉计算检查直径的一致性。区域构造需枚举 $O(m^2)$ 个交点，每个交点检验 $2m$ 个约束，故直接实现的总体计算量为 $O(m^3)$ 。

第四步（覆盖判定）：按式 (3) 计算 $\max_{P \in V} \|P - M\|$ ，判定条件为

$$\max_{P \in V} \|P - M\| \leq D/2 + \tau, \quad (4)$$

其中绝对容差 $\tau = 10^{-9} \text{ m}$ 用于容忍相切构型的浮点误差，避免相切构型被误判为不覆盖；判定不覆盖时，输出取到该最大值的顶点作为越界位置。

为保证结果不依赖具体实现，程序另有一套独立等价的区域构造算法：自一个大正方形出发，用 Sutherland–Hodgman 算法逐个半平面裁剪多边形。在五组算例上的顶点最大偏差小于 10^{-9} m ，直径枚举与旋转卡壳的结果一致，状态名与原因说明见附录 B 的 `analyse()`。

5.1.3 求解结果与分析

使用程序构造五组良态算例（检测点分布在干扰源四周，方位张角 $180^\circ \sim 249^\circ$ 、最小方位间隔不小于 25° 、与干扰源距离 $160 \sim 1300 \text{ m}$ ），结果见表 2；五组算例的定位区域直径均在 $10.59 \sim 25.64 \text{ m}$ 之间，即两次以上交会已能把候选范围从测向距离的百米量级收缩到数十米以内。表中同时给出覆盖判定，其两种结果的对照见图 3：左图（算例 A_1 ）的定位区域全部顶点都落在直径圆内或圆周上，最远顶点到圆心 M 的距离恰为 $D/2$ （比值 1.0000），此时直径圆就是该区域的最小包围圆；右图（算例 A_2 ）中，橙色虚线标出的顶点 P 落在圆外， $|PM| = 10.29 \text{ m}$ 比 $D/2 = 9.31 \text{ m}$ 大 0.98 m （比值 1.1055），直径圆覆盖判定为“否”。

表 2 问题一算例的计算结果

算例	m	顶点数	D (m)	$D/2$ (m)	$\max \ P - M\ $ (m)	比值	覆盖
A_1	6	6	17.64	8.82	8.82	1.0000	是
A_2	5	5	18.61	9.31	10.29	1.1055	否
A_3	4	5	25.64	12.82	13.75	1.0722	否
A_4	5	4	21.12	10.56	10.56	1.0000	是
A_5	6	5	10.59	5.30	5.30	1.0000	是

可见五组算例中有两组 (A_2 、 A_3) 的直径圆无法覆盖定位区域, 越界顶点到 M 的距离分别比 $D/2$ 大 0.98 m 与 0.93 m, 超出 10.55% 与 7.22%; 这两组算例的区域均为斜五边形, 由于 A_2 中两个顶点距离较近, 故可以近似为四边形, 而越界顶点所对直径形成的张角小于 90° (图 3 右图中圈出的顶点张角为 84.2°), 因而落在圆外。

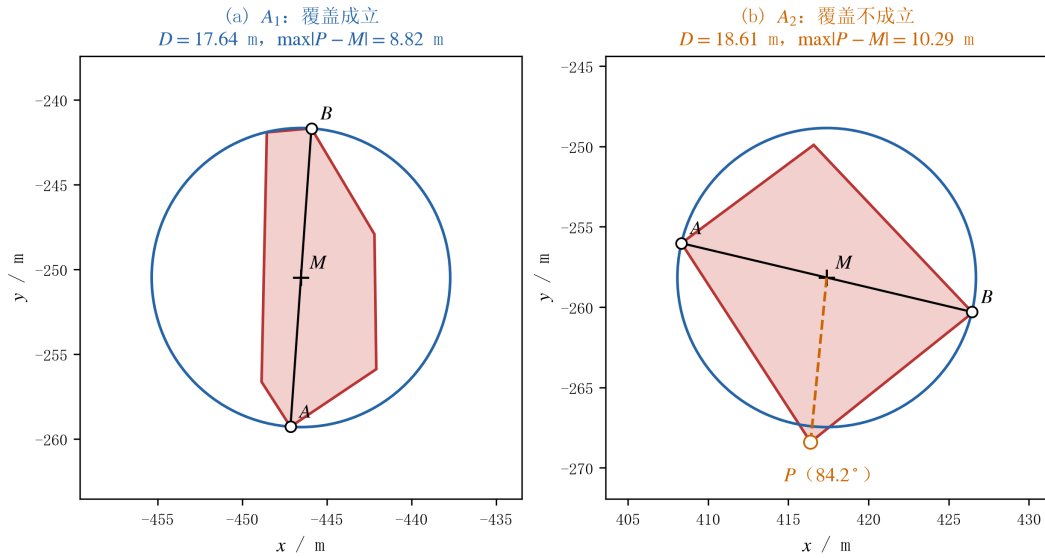


图 3 直径圆覆盖判定的两种结果: (a) 覆盖成立; (b) 覆盖不成立

表 3 四组随机扫描的覆盖失败统计 (固定种子, 理想/实际构型逐例配对)

构型	检测点数	抽样数	有界有效数	失败数	失败率	最坏比值	需放大约
理想	2	20000	20000	231	1.16%	1.0336	3.4%
理想	3 ~ 6	4000	3472	113	3.25%	1.0304	3.0%
实际	2	20000	19996	246	1.23%	1.0346	3.5%
实际	3 ~ 6	4000	3472	993	28.60%	1.6876	68.8%

注: 理想构型指各示向度精确指向源点; 实际构型指在上一批构型上叠加与算例生成同分布的 $\pm \epsilon$ 测向误差; 失败率以有界有效数为分

母, “需放大约” 为最坏比值减 1。两组 3 ~ 6 点扫描各有 528 组未满足方位间隔要求而被筛除, 不计入有界有效数。

为检验覆盖失败是否仅为个别现象, 本文以固定随机种子作四组几何扫描 (表 3)。叠加测向误差后, 3 ~ 6 个检测点的覆盖失败率由 3.25% 升至 28.60%、最坏时直径圆需放大约 69%。测向误差可能改变多边形形状并提高覆盖失败比例, 但误差不一致并不必然导致失败。上述比例针对本文规定的构型采样分布, 不能解释为所有实际布点的普遍

失败概率。其中两点扫描用于检验纯角楔几何性质，未按有效接收半径筛选检测点。本节数值的数据清单与复现命令见附录 A。

5.2 问题二：第二检测点的选择模型

5.2.1 模型建立

记号与概率描述

S_1 置于原点，其示向度方向取为 x 轴正方向； S_2 置于 (a, b) 。已知 S_1 的示向度后，源点 T 可能位置 (x_T, y_T) 服从一个条件概率分布

$$T \sim P_T(T), \quad (5)$$

而给定源点位置与 S_2 的位置， θ_2 的读数由于误差亦不能确定，近似服从

$$\theta_2 \sim P_\theta(\theta_2 | T, S_2). \quad (6)$$

位形参数 (θ_2, a, b) 一经确定，定位区域直径便由几何关系唯一确定给出：

$$D = D(\theta_2, a, b). \quad (7)$$

于是 D 服从由几何参量 (a, b) 控制的概率分布，第二个检测点的选择问题即化为求如下二维价值函数：

$$E[D](a, b) = \iint d^2 \vec{r}_T P_T(T | S_1) \int d\theta_2 D(\theta_2, a, b) P_\theta(\theta_2 | T, S_2), \quad (8)$$

$E[D](a, b)$ 越小，在该点放置第二个检测点所得定位平均效果越好。这即为问题二求解的关键衡量指标。

5.2.2 理论求解

(a) 理论推导

式 (8) 含内外两次共计三重概率积分，直接使用不便，故作以下五条近似：

1. S_2 距 S_1 的距离 R 不太小。该近似的合理性并非天然成立： S_2 距 S_1 过近时两视线的交角趋于零，所得定位区域必然很大，这是我们不希望的，故 R 较小的区域本来就不在选择范围之内，而本问关心的待选区域附近该近似较好。
2. 忽略源点纵坐标，认为其恰好落在 x 轴上，其位置只由一个参数 x_T 描述。当 R 不小时源点纵坐标 y_T 是小量，将分布对 y_T 作展开：

$$P_\theta(\theta_2 | x_T, y_T, a, b) \sim P_\theta(\theta_2 | x_T, 0, a, b) + \frac{d}{dy_T} P_\theta \Big|_{y_T=0} y_T + o(y_T) \quad (9)$$

而在第一个概率积分中区间关于 $y_T = 0$ 对称，一阶项相互抵消，故 y_T 对 $E[D]$ 的影响是高阶小量。

3. 考虑 y_T 小量近似后所得零阶项 $P_\theta(\theta_2 | x_T, 0, a, b)$ 在扇形区间内积分，可认为先验权重满足 $P_T(x_T | S_1) \propto x_T$ ，归一化后得

$$P_T(x_T | S_1) = \frac{2x_T}{x_{max}^2}, \quad x_T \in [0, x_{max}], \quad (10)$$

其中 x_{max} 为干扰源的最大可被有效接收距离，在本问中可直接取为 1500m。

4. 与第 2 条同理，由于检测点二实际测得方向角在理论值 $\theta_{2,ideal}$ 左右 1° 区间内均匀分布，偏离值为小量认为其分布近似 $\theta_2 \sim \delta(\theta_2 - \theta_{2,ideal})$ ，即等效直接认为第二次探测所得方位正对源点。
5. 在计算第二检测点的定位区域直径期望时忽略探测半径的影响，而将第二检测点的定位区域角楔延伸到足够长进行计算。这条近似在第二检测点距离第一检测点定位区域不太远时成立。因为此时整个第一检测点的定位区域完全包含在第二检测点的定位区域内，第二检测点的探测半径对定位区域直径的影响可以忽略。

五条近似把价值函数精简为单个一维积分：

$$E[D](a, b) = \int_0^{x_{max}} dx_T P_T(x_T | S_1) D(x_T, a, b). \quad (11)$$

(b) 定位区域直径的解析推导

近似取定 θ_2 正对源点后，两条探测视线在源点处的交角 α 满足

$$\sin \alpha = \frac{|b|}{r_{2T}}, \quad r_{2T} = \sqrt{(x_T - a)^2 + b^2}, \quad (12)$$

图 4 为一个特定情况下的两个检测点示意图，并取源点 T 、第二检测点 S_2 ；图中蓝、黄两个区域分别为两次探测各自的定位区域， $\angle S_1 T S_2 = \alpha$ 为两视线在源点处的交角，右上角为两区域相交所得的定位区域 $ABCD$ 的局部放大图。所得定位区域为局部图中的红色近似平行四边形 $ABCD$ ：该区域的直径即对角线 $|BD|$ 。

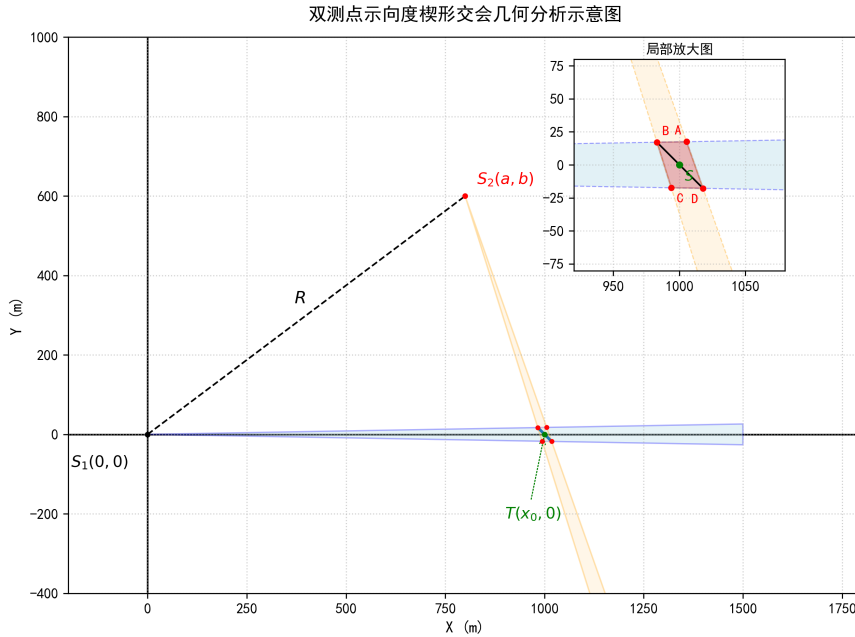


图 4 两探测束交会产生定位区域示意图

本文探测束张角极小 ($\Delta\theta = 2\varepsilon = 2^\circ \approx 0.0349 \text{ rad} \ll 1$)，故在源点处两次探测束的横向宽度分别为

$$w_1 = x_T \Delta\theta, \quad w_2 = r_{2T} \Delta\theta = \sqrt{(x_T - a)^2 + b^2} \Delta\theta. \quad (13)$$

由式 (12) 及 $(x_T - a)/r_{2T}$ 的符号，交角还满足

$$\cos \alpha = \frac{x_T - a}{r_{2T}} = \frac{x_T - a}{\sqrt{(x_T - a)^2 + b^2}}. \quad (14)$$

其对角线长度由两组对边的宽度与夹角 α 完全确定：

$$D(x_T, a, b) = \frac{\sqrt{w_1^2 + w_2^2 + 2w_1w_2 \cos \alpha}}{\sin \alpha}. \quad (15)$$

该式有直接的几何解释：下标 1、2 的两项分别对应两次探测各自的束宽，交叉项 $2w_1w_2 \cos \alpha$ 则来自两组对边之间的夹角。

把式 (13)、(12) 与 (14) 代入式 (15)，即可消去 w_1, w_2, α ，将直径完全显式化为 a, b, x_T 的函数。分子先提出公因子 $r_{2T}\Delta\theta$ ，被开方式化为 $x_T^2 + r_{2T}^2 + 2x_T(x_T - a) = 4x_T^2 - 4ax_T + a^2 + b^2$ ，于是

$$\begin{aligned} D(x_T, a, b) &= \frac{\Delta\theta r_{2T}}{|b|} \sqrt{x_T^2 + r_{2T}^2 + 2x_T(x_T - a)} \\ &= \frac{\Delta\theta}{|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}, \end{aligned} \quad (16)$$

其中第一个根式即式 (12) 中的 $r_{2T} = |S_{2T}|$ ，第二个根式是源点到点 $(a/2, -b/2)$ 之距离的两倍。定位区域直径由这两个几何量的乘积与 $|b|$ 之比给出。

式 (16) 的合理性可由两个极限情形检验：当第二个检测点正落在第一个视线方向附近 ($a \rightarrow x_T, b \rightarrow 0$) 时交角趋于 90° ，此时 $D \rightarrow \Delta\theta x_T = w_1$ ，恰为定位区域沿垂向的宽度；当 $b \rightarrow 0$ 且 $x_T \neq a$ 时两视线共线、 $\sin \alpha \rightarrow 0$ ， $D \rightarrow \infty$ ，与式 (12) 的退化判据一致。

代入 $\Delta\theta = 2^\circ = \pi/90 \text{ rad}$ ，得直径函数的解析表达式

$$D(x_T, a, b) = \frac{\pi}{90|b|} \sqrt{(x_T - a)^2 + b^2} \sqrt{(2x_T - a)^2 + b^2}. \quad (17)$$

再代回式 (11) 并利用式 (10)，即得可直接用于数值积分与优化的目标函数

$$E[D](a, b) = \frac{\pi}{45 x_{max}^2 |b|} \int_0^{x_{max}} x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (18)$$

引入无量纲变量 $u = x_T/x_{max} \in [0, 1]$ 与 $\hat{a} = a/x_{max}$ 、 $\hat{b} = b/x_{max}$ ，式 (18) 还可写成

$$E[D](a, b) = \frac{\pi x_{max}}{45 |\hat{b}|} \int_0^1 u \sqrt{[(u - \hat{a})^2 + \hat{b}^2][(2u - \hat{a})^2 + \hat{b}^2]} du, \quad (19)$$

即 $E[D]$ 只依赖 x_{max} 与无量纲位形 $(\hat{a}, \hat{b})$ ，与绝对尺度无关。

(c) 价值函数的闭合解

价值函数的不定积分核为

$$\int x_T \sqrt{[(x_T - a)^2 + b^2][(2x_T - a)^2 + b^2]} dx_T. \quad (20)$$

作平移 $t = x_T - a/2$ (即 $x_T = t + a/2$)，两个二次因子化为关于 t 的偶函数

$$(x_T - a)^2 + b^2 = \left(t - \frac{a}{2}\right)^2 + b^2, \quad (2x_T - a)^2 + b^2 = 4t^2 + b^2, \quad (21)$$

即根号内是 t 的**四次多项式** (偶四次)。因此式 (20) 的原函数属于椭圆积分类型，不能由初等函数显示表示。

我们使用 Mathematica 软件对理论公式 (18) 进行了计算绘图，计算结果见 5.2.4 节。

5.2.3 数值方法求解

为了与由理论近似解析式得到的 $E[D](a, b)$ 交叉验证, 我们使用 Python 程序对该分布作了数值求解。算法输入为已知示向度方向 (用于建立坐标系)、有效接收半径 x_{max} 与误差界 ε ; 输出为定位区域直径期望的分布 $E[D](a, b)$ 最优相对位形 (a^*, b^*) 、最小期望直径 $E[D]_{\min}$ 与候选区域, 分四步。

第一步建立标准坐标系: S_1 置于原点, 其示向度方向取为 x 轴正向, $S_2 = (a, b)$; 于是几何量 $|b|$ 、 r_{2T} 、 α 都可用 a, b, x_T 写出。

第二步 D 的精确计算: 对给定 (a, b, x_T) , 作 S_1 的两条边界射线 (与 x 轴成 $\pm\varepsilon$) 与 S_2 的两条边界射线 (绕 “ $S_2 \rightarrow T$ ” 方向成 $\pm\varepsilon$)。四条直线两两求交得 6 个候选点, 再用角楔定义筛出真正落在两个楔形内的顶点 (良态下恰为 4 个, 即四边形 $ABCD$), 取

$$D(x_T, a, b) = \max_{P, Q \in \{A, B, C, D\}} |PQ|; \quad (22)$$

两条边界线平行 (两视线共线) 时交会区域无界, 该源位样本直接舍弃。这一步用的是四边形的严格几何, 与前文解析式的平行四边形近似无关。

第三步求期望。源位采样取在 x 轴正半轴上, 上限为有效接收半径上界, 在 $[0, x_{max}]$ 上取 n 个分点, 以 $w_k \propto x_T^{(k)}$ (即式 (10)) 加权。要注意只有 “第二个检测点也能收到该频道信号” 的源位样本才是可用的: 对每个样本还须满足 $|S_2 G_k| \leq x_{max}$ 。把不满足该条件或无界的样本剔除后, 在**剩余样本**上重新归一, 得

$$E[D](a, b) = \frac{\sum_{k \in \mathcal{K}(a, b)} w_k D(x_T^{(k)}, a, b)}{\sum_{k \in \mathcal{K}(a, b)} w_k}, \quad w_k \propto x_T^{(k)}, \quad (23)$$

其中 $\mathcal{K}(a, b)$ 为可用样本集合。式 (23) 是条件期望 $E[D | \text{两次探测均有效}]$, 而非对所有源位的无条件平均; 这一区别在 S_2 远离 S_1 时不可忽略, 故本文一律采用式 (23)。

第四步寻优: 网格取以 x_{max} 为半径的靶区圆域、步长 30 m, 剔除 $|y| \leq 60$ m 的近轴退化带 (该带内两视线近简并、交会区域被拉成极窄长条甚至无界), 其余格点逐点代入式 (23), 取最小值点即 (a^*, b^*) 。为避免近轴期望值发散区域干扰绘图颜色映射, 程序中把 $E[D]$ 超过全体可用格点中位数 10 倍的格点标注为不可用带, 不参与寻优。

5.2.4 求解结果与分析

我们取 $x_{max} = 1500$ m、 $\varepsilon = 1^\circ$, 通过上述两套相互独立的计算方法分别得到结果, 并在最后据此确定第二检测点的待选区域。

(a) 理论解计算结果

按照 5.2.2 节分析, 我们使用 Mathematica 软件进行计算, 将式 (19) 的闭式被积函数, 把式 (17) 的直径代入对源位的加权积分, 在 15 m 步长的矩形网格上直接作高密度求积, 计算网格覆盖 $a \in [-500, 1795]$ m、 $b \in [-1800, 1800]$ m。图 5(a) 给出 $E[D](a, b)$ 在所在区域内的函数图像。其中 $a < -500$ 区域不在计算处理范围内, 绘制为白色。而接近 x 轴部分 $E[D]$ 发散, 超出绘图范围与机器精度限制, 亦绘制为白色。

(b) 数值模拟计算结果

我们按 5.2.3 节的算法, 使用 Python 程序, 对每个格点由两个角楔的严格几何求出交会四边形直径, 再对源位加权求积绘制了 $E[D](a, b)$ 图像; 网格为以 x_{max} 为半径的

靶区圆域、步长 30 m。由于在数值模拟中并未采取理论推导时的近似，故绘图区域限制更大。 $|S_2S_1| > x_{max}$ 或 $|S_2G| > x_{max}$ 导致两次探测不可能同时有效接收的不可行区，与两视线近似共线、 $E[D]$ 超出色标上限而被截断的区域均绘制为白色

图 5(b) 给出同一价值函数由数值模拟得到的分布。与 (a) 相比，两者的整体形态、极小值位置与色带走向完全一致，极小值为 $E[D]_{\min} = 57.34 \text{ m}$ ，位于 $(1080, \pm 660) \text{ m}$ 。

(c) 结果交叉对比

图 5 中的两幅图绘制在参数与颜色映射完全相同的两个坐标系中。直观可见在二者定义域重叠范围内，计算结果的整体形态吻合度极高。

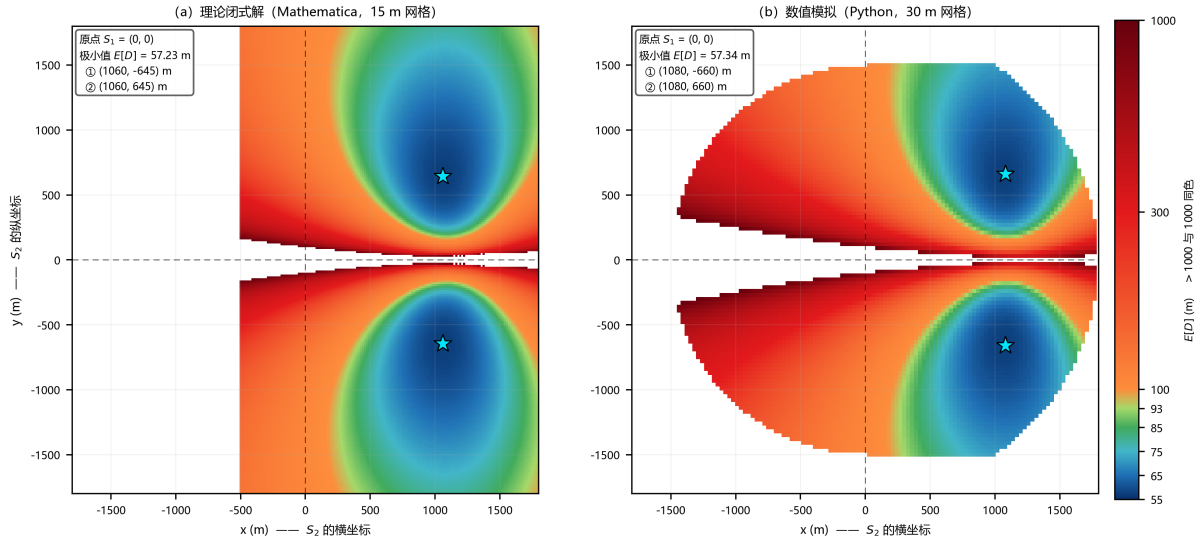


图 5 期望定位区域直径的分布：理论闭式解与数值模拟的并列对比。(a) Mathematica 按解析式在 15 m 网格上求积；(b) Python 按交会四边形严格几何在 30 m 网格上求解。两图的极小值位置与色带走向一致

为进一步定量分析，我们把 MMA 的 15 m 网格结果双线性插值到数值模拟的 30 m 格点上，对两者直接逐点对比。表 4 列出若干代表性位置的结果。在最优带 ($E[D] \leq 60 \text{ m}$ 区域) 内 374 个可对比点上，两者的相对偏差中位数为 0.14%、最大不超过 0.28%；放宽到 $E[D] \leq 100 \text{ m}$ 的 3302 个点上，相对偏差中位数为 0.18%、最大不超过 0.51%。两条完全独立的求解路线在关心的取值范围内完美吻合，说明 5.2.2 节的解析推导、5.2.3 节的四边形几何与其数值自洽自洽性。

在两种计算结果中观察 $E[D]$ 图像性质，其取值从 57 m 一直延伸到色标上限 10^3 m 以上，且在极小值附近平缓而在 x 轴附近发散，故图中采用非线性色标：把 55 ~ 100 m 一段拉开、100 m 以上压缩，提升问题更关心的低值区的区分度。图像关于 $b = 0$ (S_1 的视线方向) 严格上下对称，极小值成对出现，这是由体系对称性所决定的。 $E[D] \leq 60 \text{ m}$ 的最优带为极小点周围的一小片区域，在此范围内靠近和远离检测点一的探测区间，都会使得最终探测区间的直径期望增大。这是由于当检测点二距离检测点一的定位区域很近时，观测角夹角正弦 $\sin \alpha$ 平均值小，使得最终定位区域重叠极大，直径快速发散。当检测点二距离检测点一的定位区域很远时，检测点二的定位区域在干扰源点处展宽已经变得较大，使得定位区域直径增大。

表 4 理论闭式解与数值模拟在代表性第二检测点位置的对比

a (m)	b (m)	理论闭式解 $E[D]$ (m)	数值模拟 $E[D]$ (m)	相对偏差
1080	660	57.256	57.336	+0.14%
1050	660	57.260	57.351	+0.16%
1110	750	57.784	57.849	+0.11%
1200	690	58.932	58.963	+0.05%
1200	600	59.194	59.222	+0.05%
1200	900	60.441	60.475	+0.06%
900	900	61.038	61.154	+0.19%
1350	750	64.434	64.406	-0.04%
900	1200	65.973	66.066	+0.14%
1410	600	69.941	69.856	-0.12%
1500	900	72.758	72.684	-0.10%
600	600	78.228	78.499	+0.35%
300	900	94.310	94.561	+0.27%

(d) 待选区域确定

问题二要求给出的是第二检测点的可选范围。综合上文计算结果，本文**选取** $E[D] \leq 60$ m 的区域作为第二检测点的待选区域：该阈值比极小值高约 5%，既保证定位效果（期望直径接近销毁半径数量级），又给现场留出足够的位形余量。具体范围见图 6 所示。两套计算给出的同一区域的边界几乎重合。实际布点时建议取区域中心附近的 $(a, b) \approx (0.71 x_{max}, 0.43 x_{max}) = (1065, 645)$ m，此时 $E[D]$ 接近极小值，在期望意义下最利于确认干扰源位置。

5.3 问题三：多源定位与清除策略

5.3.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.3.2 求解与结果

【待填充：给出该问的求解结果与实测指标。】

5.4 问题四：含定向源时的定位与清除模型

5.4.1 模型建立

【待填充：给出该问的模型与求解方法。】

5.4.2 求解与结果

【待填充：给出该问的求解结果与分析。】

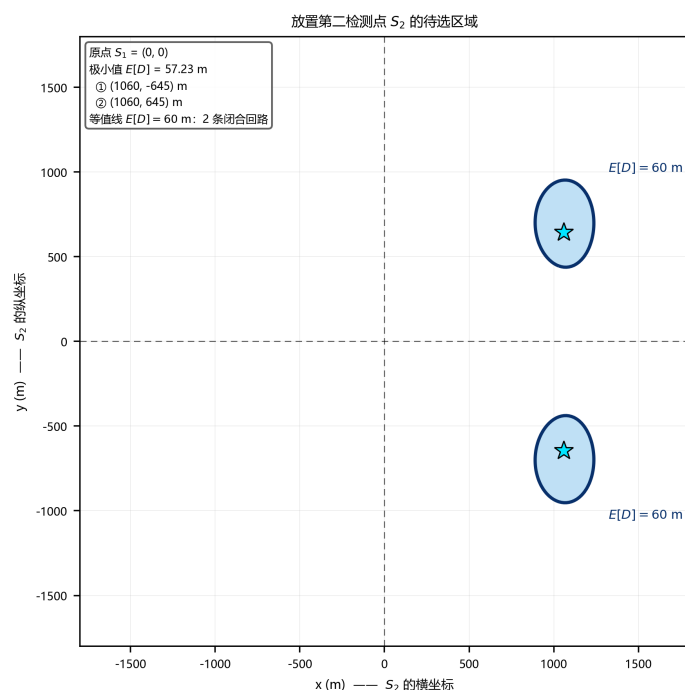


图 6 $E[D] \leq 60$ m 等值线围出的第二检测点待选区域

六、模型评价与改进

6.1 灵敏度分析

【待填充：对模型中的关键参数作灵敏度分析。】

6.2 模型优点

【待填充：完成问题二至四后统一总结四个模型的优点。】

6.3 模型缺点

【待填充：同上，统一总结缺点。】

6.4 改进方向

【待填充：统一给出改进方向。】

参考文献

- [1] 作者. 无线电测向与定位技术 [M]. 出版社, 年份.
- [2] 作者. 论文题目 [J]. 期刊名, 年份, 卷 (期): 页码.
- [3] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2026 年修订稿）.

附录 A 支撑材料文件列表

支撑材料按问题分别存放，包含计算与绘图程序、输入算例和结果数据。以下路径相对于附件根目录 B_locator/。

1. Q1/p1_intersection.py: 问题一角楔交会、直径和覆盖判定。

2. Q1/p1_experiments.py: 生成五组算例、四组固定种子扫描并写出汇总结果。
3. Q1/make_figures.py: 重绘问题一角楔示意、算法流程和覆盖对比三幅插图。
4. Q1/cases/: 五组输入 CSV 及真值核验文件; Q1/results/: 逐例结果和汇总 JSON。
5. Q2/pysimulation/src/p2_grid_expectation.py: 问题二数值模拟源程序, 交会四边形直径、加权期望、寻优与候选区域 (附录 C)。
6. Q2/MMAcode/expected_d mathematica_ver.nb: 理论闭式解的 Mathematica 程序, 导出 15 m 网格上的 $E[D]$ 与最优点。
7. Q2/pysimulation/out/p2_grid/p2_grid_map.csv、p2_grid_report.json: 数值模拟逐点结果与汇总统计 (本文数值来源)。
8. Q2/make_q2_paper_figures.py: 问题二正文插图生成脚本 (由上述两份原始数据出图)。
9. figures/p2-mma-vs-pysim.png: 理论闭式解与数值模拟的并列对比 (图 5)。
10. figures/p2-contour60.png: $E[D] \leq 60$ m 等值线围出的待选区域 (图 6)。
11. figures/p2-wedge-geometry.png: 两探测束交会与定位区域直径构造 (图 4)。
12. Q2/Q2modeling.md 等: 问题二建模手稿、参考热力图与 Mathematica 校验脚本。

问题一程序依赖 Python 3、NumPy 和 Matplotlib, 在附件根目录依次运行以下命令, 可生成五组算例、四组固定种子扫描及正文插图。

Listing 1: 问题一数据与图形的复现命令

```
python Q1/p1_experiments.py
python Q1/make_figures.py
```

附录 B 问题一完整源程序

以下代码直接从对应支撑材料文件载入, 各程序按所列文件名保存并保持目录结构即可运行。

2.1 角楔交会、直径与覆盖判定

Listing 2: Q1/p1_intersection.py

```
# -*- coding: utf-8 -*-
"""问题一几何求解。坐标单位 m, 示向度为从正东逆时针量取的角度。

命令行: python p1_intersection.py cases/p1_case01.csv
全部实验: python p1_experiments.py
"""

from __future__ import annotations
import csv
import json
import math
import os
from pathlib import Path

import numpy as np

BEARING_ERR = 1.0      # 示向度误差界 (度)
```

```

ANG_TOL = 1e-9          # 角度判据容差 (度)
DIST_TOL = 1e-6          # 半平面判据容差 (米)
COVER_TOL = 1e-9         # 覆盖判据容差 (米)
CLIP = 1e7               # 裁剪参考实现的外框半边长 (米)
DIAM_BRUTE_LIMIT = 128   # 顶点数不超过该值时直径用暴力枚举

def bearing(p, q):
    """p 处指向 q 的方位角 (度, [0,360)) """
    return math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])) % 360.0

def ang_diff(a, b):
    """a-b 归一化到 [-180,180)。"""
    return (a - b + 180.0) % 360.0 - 180.0

def wedge_halfplanes(p, theta, err=BEARING_ERR):
    """角楔 W = {X : |ang(X-p) - theta| <= err} 的两个半平面 a · X + b <= 0"""
    p = np.asarray(p, float)
    tm = math.radians((theta - err) % 360.0)
    tp = math.radians((theta + err) % 360.0)
    dm = np.array([math.cos(tm), math.sin(tm)])
    dp = np.array([math.cos(tp), math.sin(tp)])
    # cross(dm, X-p) >= 0  <=>  a1 · X + b1 <= 0
    a1 = np.array([dm[1], -dm[0]])
    b1 = float(-dm[1] * p[0] + dm[0] * p[1])
    # cross(dp, X-p) <= 0  <=>  a2 · X + b2 <= 0
    a2 = np.array([-dp[1], dp[0]])
    b2 = float(dp[1] * p[0] - dp[0] * p[1])
    return [(a1, b1), (a2, b2)]

def region_is_unbounded(svds, err=BEARING_ERR):
    """非空区域的公共延伸方向; 无公共方向时返回 None, 复杂度 O(m)。"""
    if len(svds) == 0:
        return None
    ref = float(svds[0])
    lo, hi = -err, err
    for theta in svds[1:]:
        center = ang_diff(theta, ref)
        lo, hi = max(lo, center - err), min(hi, center + err)
        if lo > hi + ANG_TOL:
            return None
    return (ref + (lo + hi) / 2.0) % 360.0

def _line_intersection(P, u, Q, v):
    """直线 P + t · u 与 Q + s · v 的交点, 平行时返回 None"""
    det = u[0] * (-v[1]) - (-v[0]) * u[1]
    if abs(det) < 1e-14:
        return None
    r = Q - P
    t = (r[0] * (-v[1]) - (-v[0]) * r[1]) / det
    return P + t * u

def convex_hull(P):
    """Andrew monotone chain, 逆时针, 去掉共线点"""
    def half(pts):
        st = []
        for p in pts:

```



```

        while len(st) >= 2 and _cross2(st[-1] - st[-2], p - st[-2]) <= 0:
            st.pop()
            st.append(p)
        return st
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    lo = half(P)
    hi = half(P[::-1])
    return np.array(lo[:-1] + hi[:-1])

def _cross2(u, v):
    return float(u[0] * v[1] - u[1] * v[0])

def _dedup_hull(cand):
    """候选顶点去重 + 凸包, 返回 (P, "empty"|"degenerate"|"polygon")"""
    if len(cand) == 0:
        return np.zeros((0, 2)), "empty"
    P = np.array(cand, float)
    P = P[np.lexsort((P[:, 1], P[:, 0]))]
    ded = [P[0]]
    for v in P[1:]:
        if np.linalg.norm(v - ded[-1]) > 1e-7:
            ded.append(v)
    P = np.array(ded)
    if len(P) < 3:
        return P, "degenerate"
    H = convex_hull(P)
    if len(H) < 3:
        return H, "degenerate"
    return H, "polygon"

def region_by_vertices(pts, svds, err=BEARING_ERR, tol=DIST_TOL):
    """主算法: 枚举 2m 条边界直线的两两交点, 用全部半平面判据筛选后求凸包。

    R 是凸集, 其任一极点必是两条起作用约束边界直线的交点, 故枚举 + 筛选是精确的;
    半平面判据在处理"边界直线过检测点"的情形时仍良定义, 检测点本身也可作顶点保留。
    """
    pts, svds = np.asarray(pts, float), np.asarray(svds, float)
    if pts.ndim != 2 or pts.shape[1] != 2 or len(pts) != len(svds) or len(pts) == 0:
        raise ValueError("需输入等长、非空的二维检测点和示向度数组")
    if not np.isfinite(pts).all() or not np.isfinite(svds).all() or not 0 < err < 90:
        raise ValueError("坐标和示向度须为有限数, 误差界须在 (0,90) 度内")
    H, lines = [], []
    for p, th in zip(pts, svds):
        H += wedge_halfplanes(p, th, err)
        for s in (-1.0, 1.0):
            a = math.radians((th + s * err) % 360.0)
            lines.append((np.asarray(p, float), np.array([math.cos(a), math.sin(a)])))
    cand = []
    n = len(lines)
    for i in range(n):
        for j in range(i + 1, n):
            X = _line_intersection(lines[i][0], lines[i][1], lines[j][0], lines[j][1])
            if X is None or not np.all(np.isfinite(X)):
                continue
            if all(float(a @ X + b) <= tol for (a, b) in H):
                cand.append(X)
    return _dedup_hull(cand)

```

```

def clip_halfplane(poly, a, b, tol=DIST_TOL):
    """Sutherland-Hodgman: 保留 {X : a * X + b <= 0} 的部分"""
    if not poly:
        return []
    out = []
    n = len(poly)
    for i in range(n):
        P = poly[i]
        Q = poly[(i + 1) % n]
        fp = float(a @ P + b)
        fq = float(a @ Q + b)
        if fp <= tol:
            out.append(P)
        if (fp < -tol and fq > tol) or (fp > tol and fq < -tol):
            t = fp / (fp - fq)
            out.append(P + t * (Q - P))
    ded = []
    for v in out:
        if not ded or np.linalg.norm(v - ded[-1]) > 1e-9:
            ded.append(v)
    if len(ded) > 1 and np.linalg.norm(ded[0] - ded[-1]) < 1e-9:
        ded.pop()
    return ded

def region_by_clipping(pts, svds, err=BEARING_ERR, box=CLIP, tol=DIST_TOL):
    """有界算例的裁剪校验: 若碰到人为外框则要求增大 box。"""
    poly = [np.array([-box, -box], float), np.array([box, -box], float),
            np.array([box, box], float), np.array([-box, box], float)]
    for p, th in zip(pts, svds):
        for (a, b) in wedge_halfplanes(p, th, err):
            poly = clip_halfplane(poly, a, b, tol)
            if not poly:
                return np.zeros((0, 2)), "empty"
    P = np.array(poly, float)
    if len(P) < 3:
        return P, "degenerate"
    if float(np.max(np.abs(P))) >= box - tol:
        raise ValueError("区域碰到校验外框: 请增大 box 或先检查有界性")
    H = convex_hull(P)
    return (H if len(H) >= 3 else P), "polygon"

def region_from_bearings(pts, svds, err=BEARING_ERR):
    """主入口: 返回 (顶点数组, status), status 取 empty / unbounded / degenerate / bounded。"""
    P, st = region_by_vertices(pts, svds, err)
    if st == "empty":
        return P, "empty"
    rec = region_is_unbounded(list(svds), err) if len(svds) else None
    if rec is not None:
        return P, "unbounded"
    if st == "degenerate":
        return P, "degenerate"
    return P, "bounded"

def _diameter_brute(P):
    n = len(P)
    best, pair = -1.0, (P[0], P[1])
    for i in range(n - 1):
        d = np.linalg.norm(P[i + 1:] - P[i], axis=1)
        k = int(np.argmax(d))

```

```

        if float(d[k]) > best:
            best, pair = float(d[k]), (P[i], P[i + 1 + k])
    return best, pair

def rotating_calipers(P):
    """凸多边形直径（旋转卡壳，O(n)）。P 需为逆时针凸序（允许共线点）。"""
    n = len(P)
    if n < 3:
        return diameter(P, brute_limit=2)
    k = 1
    best, pair = -1.0, (P[0], P[1])
    for i in range(n):
        i2 = (i + 1) % n
        e = P[i2] - P[i]
        while True:
            k2 = (k + 1) % n
            if abs(_cross2(e, P[k2] - P[i])) > abs(_cross2(e, P[k] - P[i])):
                k = k2
            else:
                break
        for j in (k, (k + 1) % n):
            d = float(np.linalg.norm(P[i] - P[j]))
            if d > best:
                best, pair = d, (P[i], P[j])
            d = float(np.linalg.norm(P[i2] - P[j]))
            if d > best:
                best, pair = d, (P[i2], P[j])
    return best, pair

def diameter(P, brute_limit=DIAM_BRUTE_LIMIT):
    """凸多边形直径，返回 (D, (A, B))"""
    n = len(P)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (P[0], P[0])
    if n == 2:
        return float(np.linalg.norm(P[0] - P[1])), (P[0], P[1])
    if n <= brute_limit:
        return _diameter_brute(P)
    return rotating_calipers(P)

def min_enclosing_circle(P):
    """最小包围圆 (center, radius)；顶点数很少，直接枚举 1/2/3 点候选圆。"""
    P = np.asarray(P, float)
    n = len(P)
    if n == 0:
        return None, 0.0
    if n == 1:
        return P[0].copy(), 0.0
    cand_s = [(P[i].copy(), 0.0) for i in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            c = 0.5 * (P[i] + P[j])
            cand_s.append((c, float(np.linalg.norm(P[i] - c))))
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                A, B, C = P[i], P[j], P[k]

```

```

        d = 2.0 * (A[0] * (B[1] - C[1]) + B[0] * (C[1] - A[1])
                  + C[0] * (A[1] - B[1]))
        if abs(d) < 1e-12:
            continue
        aa, bb, cc = float(A @ A), float(B @ B), float(C @ C)
        ux = (aa * (B[1] - C[1]) + bb * (C[1] - A[1]) + cc * (A[1] - B[1])) / d
        uy = (aa * (C[0] - B[0]) + bb * (A[0] - C[0]) + cc * (B[0] - A[0])) / d
        c = np.array([ux, uy])
        cand_s.append((c, float(np.linalg.norm(A - c))))

    best = None
    for c, r in cand_s:
        if all(float(np.linalg.norm(P[i] - c)) <= r + 1e-9 for i in range(n)):
            if best is None or r < best[1]:
                best = (c, r)
    return best

def region_area(P):
    n = len(P)
    if n < 3:
        return 0.0
    return abs(0.5 * float(np.sum(P[:, 0] * np.roll(P[:, 1], -1)
                                   - np.roll(P[:, 0], -1) * P[:, 1])))

def analyse(pts, svds, err=BEARING_ERR):
    """给定检测点与示向度，输出定位区域的全套几何量（长度量按 4 位小数落盘）。"""
    P, status = region_from_bearings(pts, svds, err)
    base = {"n_bearings": len(pts), "status": status}
    if status == "empty":
        base["reason"] = "各角楔无公共交集，测量数据与误差界不相容"
        return base
    if status == "unbounded":
        rec = region_is_unbounded(list(svds), err)
        base["reason"] = "非空角楔交集存在公共延伸方向，直径为无穷大"
        base["recession_direction_deg"] = None if rec is None else round(float(rec), 4)
        base["n_vertices"] = int(len(P))
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base
    if status == "degenerate" or len(P) < 3:
        base["status"] = "degenerate"
        base["reason"] = "公共交集退化为线段或点"
        base["vertices"] = [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P]
        return base

    D, (A, B) = diameter(P)
    c = 0.5 * (A + B)
    maxr = float(max(np.linalg.norm(v - c) for v in P))
    worst = P[int(np.argmax(np.linalg.norm(P - c, axis=1)))]
    mc, mr = min_enclosing_circle(P)
    base.update({
        "n_vertices": int(len(P)),
        "vertices": [[round(float(v[0]), 4), round(float(v[1]), 4)] for v in P],
        "diameter_m": round(D, 4),
        "diameter_endpoints": [[round(float(A[0]), 4), round(float(A[1]), 4)],
                               [round(float(B[0]), 4), round(float(B[1]), 4)]],
        "circle_center": [round(float(c[0]), 4), round(float(c[1]), 4)],
        "circle_radius_m": round(D / 2.0, 4),
        "max_vertex_dist_m": round(maxr, 4),
        "diameter_circle_covers": bool(maxr <= D / 2.0 + COVER_TOL),
        "farthest_vertex": [round(float(x), 4) for x in worst],
        "min_enclosing_center": [round(float(mc[0]), 4), round(float(mc[1]), 4)],
    })

```

```

        "min_enclosing_radius_m": round(float(mr), 4),
        "min_circle_over_half_diameter": round(float(mr) / (D / 2.0), 6),
        "area_m2": round(region_area(P), 4),
    })
    return base

def run_case_csv(path, truth_path=None, err=BEARING_ERR):
    """读单个算例 CSV (列含 x_m,y_m,svd_deg) 与可选真值 JSON, 返回 analyse 结果。"""
    with open(path, encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
    svds = [float(r["svd_deg"]) for r in rows]
    res = analyse(pts, svds, err)
    res["n_bearings"] = len(rows)
    if truth_path and os.path.exists(truth_path):
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        G = truth["source_xy"]
        res["truth_xy"] = G
        if "circle_center" in res:
            res["center_error_vs_truth_m"] = round(math.dist(res["circle_center"], G), 3)
        if "min_enclosing_center" in res:
            res["mincircle_error_vs_truth_m"] = round(
                math.dist(res["min_enclosing_center"], G), 3)
        # 真值是否满足全部约束 (等价于在半平面内)
        H = []
        for p, th in zip(pts, svds):
            H += wedge_halfplanes(p, th, err)
        res["truth_inside_region"] = bool(all(a @ np.asarray(G, float) + b <= 1e-6
                                             for (a, b) in H))
    return res

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("csv", nargs="?", default="cases/p1_case01.csv")
    parser.add_argument("--error", type=float, default=BEARING_ERR)
    args = parser.parse_args()
    path = Path(args.csv)
    if not path.is_absolute() and not path.exists():
        path = Path(__file__).resolve().parent / path
    print(json.dumps(run_case_csv(path, path.with_suffix(".truth.json"), args.error),
                    ensure_ascii=False, indent=2))

```

2.2 算例生成、随机扫描与几何校验

Listing 3: Q1/p1_experiments.py

```

# -*- coding: utf-8 -*-
"""问题一：固定种子生成五组算例，求解并统计四组几何扫描。

运行 python p1_experiments.py: 数据写入 cases/ 和 results/。
加 --skip-sweep 只计算算例，使用独立结果名，避免覆盖完整扫描。
"""
from __future__ import annotations
import argparse
import csv
import json

```

```

import math
import os
import random
import sys

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import p1_intersection as P

R_ARENA = 1800.0 # 目标区域半径 (m)
R_RECV_MIN, R_RECV_MAX = 1000.0, 1500.0 # 有效接收距离范围 (m)
NEAR_R = 5.0 # 近场阈值 (m)
DEFAULT_SEED = 20260901
SWEEP_TWO_POINT = {"n_trials": 20000, "seed": 17, "r_max": 1800.0}
SWEEP_WELL_COND = {"n_trials": 4000, "seed": 5, "min_gap": 3.0,
                    "dist": (300.0, 1500.0), "m_choices": (3, 4, 5, 6)}
SWEEP_ERR_SEED_OFFSET = 900001 # 误差随机流与构型随机流分离, 保证理想/实际同构型配对
CASE_FIELDS = ["det_id", "x_m", "y_m", "svd_deg", "true_svd_deg",
               "true_az_err_deg", "dist_m"]
CASE_LABELS = ["A1", "A2", "A3", "A4", "A5"]

def wrap360(a):
    return a % 360.0

def bearing(p, q):
    """p 处指向 q 的方位角 (度)"""
    return wrap360(math.degrees(math.atan2(q[1] - p[1], q[0] - p[0])))

def in_cone(src, p, half_deg):
    """点 p 是否落在以 src 为顶点、沿定向方向的 ±half_deg 锥内 (全向 half=180)"""
    if half_deg >= 180.0:
        return True
    return abs(P.ang_diff(bearing(src["xy"], p), src["dir_deg"])) <= half_deg + 1e-12

def measure(src, p):
    """按题目附录规则返回一次检测结果 (不含示向度误差)"""
    d = math.dist(p, src["xy"])
    if d > src["r_recv"]:
        return "no_signal", None
    if not in_cone(src, p, src["cone_half"]):
        return "no_signal", None
    if d <= NEAR_R:
        return "near", None
    return "direction", bearing(p, src["xy"])

def make_case_p1(rng, n_pts=3, min_az_gap=25.0, min_dist=150.0,
                max_dist=1300.0, spread=140.0):
    """生成一组"良态"算例: 检测点分布在干扰源四周一定方位张角内, 且有最小间隔与距离。"""
    ang = rng.uniform(0, 2 * math.pi)
    rad = rng.uniform(50, 0.75 * R_ARENA)
    G = (rad * math.cos(ang), rad * math.sin(ang))
    src = {"xy": G, "r_recv": rng.uniform(R_RECV_MIN, R_RECV_MAX),
           "cone_half": 180.0, "dir_deg": None}

    n = max(2, n_pts)
    base = rng.uniform(0, 360)

```

```

span = 2 * spread
while n * min_az_gap > span:
    min_az_gap *= 0.85
offs = np.sort(rng.uniform(-spread, spread, size=n))
for _ in range(200):
    gaps = np.diff(offs)
    if len(gaps) == 0 or gaps.min() >= min_az_gap:
        break
    offs = np.sort(rng.uniform(-spread, spread, size=n))

pts, azs = [], []
for o in offs:
    a = math.radians((base + o) % 360.0)
    for _try in range(80):
        d = rng.uniform(min_dist, min(max_dist, src["r_rcv"]))
        p = (G[0] + d * math.cos(a), G[1] + d * math.sin(a))
        if math.hypot(*p) <= R_ARENA:
            pts.append(p)
            azs.append(bearing(p, G))
            break
if len(pts) < 2:
    return [], {"error": "failed to build well-conditioned case"}

errs = rng.uniform(-P.BEARING_ERR, P.BEARING_ERR, size=len(pts))
rows = []
for i, (p, e) in enumerate(zip(pts, errs), start=1):
    res, svd = measure(src, p)
    if res != "direction":
        continue
    rows.append({
        "det_id": i,
        "x_m": round(p[0], 3),
        "y_m": round(p[1], 3),
        "svd_deg": round(wrap360(svd + e), 2),
        "true_svd_deg": round(svd, 4),
        "true_az_err_deg": round(e, 4),
        "dist_m": round(math.dist(p, G), 3),
    })
truth = {"source_xy": [round(G[0], 3), round(G[1], 3)],
        "r_rcv_m": round(src["r_rcv"], 3),
        "type": "omnidirectional", "dir_deg": None,
        "bearing_error_bound_deg": P.BEARING_ERR,
        "azimuth_span_deg": round(float(max(azs) - min(azs)), 2),
        "min_azimuth_gap_deg": round(float(np.min(np.diff(np.sort(azs)))), 2),
        "if len(azs) > 1 else None,
        "n_detection_points": len(rows)}
return rows, truth

def write_csv(path, rows, fields):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=fields, lineterminator="\n")
        w.writeheader()
        w.writerows(rows)

def gen_cases(out_dir, seed=DEFAULT_SEED, n_cases=5):
    """生成 n_cases 组算例，返回（文件路径，真值）列表。"""
    rng = np.random.default_rng(seed)
    out = []
    for k in range(n_cases):
        rows, truth = make_case_p1(rng, n_pts=int(rng.integers(3, 7)))

```

```

        name = "p1_case%02d" % (k + 1)
        csv_path = os.path.join(out_dir, name + ".csv")
        truth_path = os.path.join(out_dir, name + ".truth.json")
        write_csv(csv_path, rows, CASE_FIELDS)
        with open(truth_path, "w", encoding="utf-8") as f:
            json.dump(truth, f, ensure_ascii=False, indent=2)
        out.append((csv_path, truth_path))
    return out

def solve_full(pts, svds):
    """返回 dict: status / D / half / max_vertex_dist / ratio / excess / covers。"""
    V, st = P.region_from_bearings(pts, svds)
    if st != "bounded":
        return {"status": st}
    D, (A, B) = P.diameter(V)
    M = 0.5 * (A + B)
    dists = np.linalg.norm(V - M, axis=1)
    k = int(np.argmax(dists))
    maxr = float(dists[k])
    half = D / 2.0
    ua, ub = A - V[k], B - V[k]
    na, nb = float(np.linalg.norm(ua)), float(np.linalg.norm(ub))
    # 覆盖成立时最远顶点就是直径端点 (ua 或 ub 为零向量), 该处的张角无定义
    angle = None
    if na > 1e-12 and nb > 1e-12:
        cosang = float(ua @ ub / (na * nb))
        angle = math.degrees(math.acos(max(-1.0, min(1.0, cosang))))
    return {"status": "bounded", "D": float(D), "half": float(half),
            "max_vertex_dist": maxr, "ratio": maxr / half,
            "excess": maxr - half, "covers": bool(maxr <= half + P.COVER_TOL),
            "worst_vertex_angle_deg": angle}

def run_cases(case_paths):
    """逐例求解 (analyse 落盘 4 位小数 + 全精度比值与张角)。"""
    results = []
    for i, (csv_path, truth_path) in enumerate(case_paths):
        res = P.run_case_csv(csv_path, truth_path)
        res["case"] = os.path.basename(csv_path)
        res["label"] = CASE_LABELS[i] if i < len(CASE_LABELS) else "C%d" % (i + 1)
        if res["status"] == "bounded":
            rows = list(csv.DictReader(open(csv_path, encoding="utf-8")))
            pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
            svds = [float(r["svd_deg"]) for r in rows]
            full = solve_full(pts, svds)
            res["ratio"] = round(full["ratio"], 6)
            res["excess_m"] = round(full["excess"], 4)
            res["excess_pct"] = round(100.0 * full["excess"] / full["half"], 4)
            ang = full["worst_vertex_angle_deg"]
            res["worst_vertex_angle_deg"] = None if ang is None else round(ang, 2)
        results.append(res)
    return results

def sweep_two_point(n_trials=SWEEP_TWO_POINT["n_trials"], seed=SWEEP_TWO_POINT["seed"],
                    r_max=SWEEP_TWO_POINT["r_max"], with_error=False):
    """两测点构型: 检测点落在半径 r_max 的目标区域内, 方位差 3°~179°, 只统计有界者。

    with_error=False 为理想构型 (示向度精确指向源点); True 为实际构型 (在同一批构型上
    叠加 ±BEARING_ERR 的测向误差)。误差取自独立随机流, 两组构型逐例配对。
    """

```



```

rng = random.Random(seed)
erng = random.Random(seed + SWEEP_ERR_SEED_OFFSET)
n_bounded = n_fail = 0
n_unbounded = n_empty = n_degenerate = n_rejected = 0
worst = None
for _ in range(n_trials):
    dth = rng.uniform(3.0, 179.0)
    th1 = rng.uniform(90.0, 270.0)
    th2 = (th1 + dth) % 360.0
    r1, r2 = rng.uniform(150.0, r_max), rng.uniform(150.0, r_max)
    A = (r1 * math.cos(math.radians(th1 + 180.0)),
          r1 * math.sin(math.radians(th1 + 180.0)))
    B = (r2 * math.cos(math.radians(th2 + 180.0)),
          r2 * math.sin(math.radians(th2 + 180.0)))
    if math.hypot(*A) > r_max or math.hypot(*B) > r_max:
        n_rejected += 1
        continue
    if with_error:
        # 测得示向度 = 真方位角 + 误差
        th1 = (th1 + erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)) % 360.0
        th2 = (th2 + erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)) % 360.0
    st = solve_full([A, B], [th1, th2])
    if st["status"] != "bounded":
        if st["status"] == "unbounded":
            n_unbounded += 1
        elif st["status"] == "empty":
            n_empty += 1
        elif st["status"] == "degenerate":
            n_degenerate += 1
        continue
    n_bounded += 1
    if worst is None or st["ratio"] > worst["ratio"]:
        worst = {"S1": [A[0], A[1], th1],
                  "S2": [B[0], B[1], th2],
                  "D_m": round(st["D"], 4), "max_vertex_dist_m": round(st["max_vertex_dist"], 4),
                  "ratio": round(st["ratio"], 6), "covers": st["covers"]}
    if not st["covers"]:
        n_fail += 1
return {"n_trials": n_trials, "seed": seed, "r_max_m": r_max,
        "with_error": bool(with_error),
        "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
        "n_bounded": n_bounded, "n_fail": n_fail,
        "n_unbounded": n_unbounded, "n_empty": n_empty, "n_degenerate": n_degenerate,
        "n_rejected_geometry": n_rejected,
        "fail_pct": 100.0 * n_fail / max(n_bounded, 1),
        "worst_ratio": None if worst is None else worst["ratio"],
        "worst_config": worst}

def sweep_well_conditioned(n_trials=SWEEP_WELL_COND["n_trials"],
                           seed=SWEEP_WELL_COND["seed"],
                           min_gap=SWEEP_WELL_COND["min_gap"],
                           dist=SWEEP_WELL_COND["dist"],
                           m_choices=SWEEP_WELL_COND["m_choices"],
                           with_error=False):
    """3-6 个检测点的良态构型：方位角最小间隔 > min_gap, 检测点到原点距离在 dist 内。

    with_error 的含义同 sweep_two_point: True 时在同一批构型上叠加 ±BEARING_ERR 测向误差。
    """
    rng = random.Random(seed)
    erng = random.Random(seed + SWEEP_ERR_SEED_OFFSET)
    n_bounded = n_fail = 0
    n_unbounded = n_empty = n_degenerate = n_rejected = 0

```

```

per_m = {str(m): {"bounded": 0, "fail": 0} for m in m_choices}
worst = None
for _ in range(n_trials):
    m = rng.choice(list(m_choices))
    angs = sorted(rng.uniform(0.0, 360.0) for _ in range(m))
    gaps = [min((angs[i] - angs[j]) % 360.0, (angs[j] - angs[i]) % 360.0)
             for i in range(m) for j in range(i + 1, m)]
    if min(gaps) <= min_gap:
        n_rejected += 1
        continue
    pts, svds = [], []
    for t in angs:
        rr = rng.uniform(dist[0], dist[1])
        pts.append((rr * math.cos(math.radians(t)), rr * math.sin(math.radians(t))))
        s = t + 180.0
        if with_error:
            # 测得示向度 = 真方位角 + 误差
            s += erng.uniform(-P.BEARING_ERR, P.BEARING_ERR)
        svds.append(s % 360.0)
    st = solve_full(pts, svds)
    if st["status"] != "bounded":
        if st["status"] == "unbounded":
            n_unbounded += 1
        elif st["status"] == "empty":
            n_empty += 1
        elif st["status"] == "degenerate":
            n_degenerate += 1
        continue
    n_bounded += 1
    per_m[str(m)]["bounded"] += 1
    if worst is None or st["ratio"] > worst["ratio"]:
        worst = {"m": m, "pts": [list(p) for p in pts],
                  "svds_deg": list(svds),
                  "D_m": round(st["D"], 4), "max_vertex_dist_m": round(st["max_vertex_dist"], 4),
                  "ratio": round(st["ratio"], 6), "covers": st["covers"]}
    if not st["covers"]:
        n_fail += 1
        per_m[str(m)]["fail"] += 1
for v in per_m.values():
    v["fail_pct"] = round(100.0 * v["fail"] / max(v["bounded"], 1), 4)
return {"n_trials": n_trials, "seed": seed, "min_gap_deg": min_gap,
        "dist_range_m": list(dist), "m_choices": list(m_choices),
        "with_error": bool(with_error),
        "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
        "n_bounded": n_bounded, "n_fail": n_fail,
        "n_unbounded": n_unbounded, "n_empty": n_empty, "n_degenerate": n_degenerate,
        "n_rejected_geometry": n_rejected,
        "fail_pct": 100.0 * n_fail / max(n_bounded, 1),
        "worst_ratio": None if worst is None else worst["ratio"],
        "worst_config": worst, "per_m": per_m}

SWEEP_PARAM_KEYS = ("n_trials", "seed", "r_max_m", "min_gap_deg", "dist_range_m",
                    "m_choices", "with_error", "bearing_error_deg")

def build_summary(case_paths, results, sweeps, seed):
    ok = [r for r in results if r["status"] == "bounded"]
    cases_summary = {
        "n_cases": len(results),
        "n_bounded": len(ok),
        "n_unbounded": sum(1 for r in results if r["status"] == "unbounded"),
        "n_degenerate": sum(1 for r in results if r["status"] == "degenerate"),
    }

```

```

        "n_empty": sum(1 for r in results if r["status"] == "empty"),
        "diameter_m_range": [min(r["diameter_m"] for r in ok),
                              max(r["diameter_m"] for r in ok)] if ok else None,
        "n_diameter_circle_misses": sum(1 for r in ok if not r["diameter_circle_covers"]),
        "min_circle_over_half_diameter_max": max(
            r["min_circle_over_half_diameter"] for r in ok) if ok else None,
        "n_truth_inside_region": sum(1 for r in ok if r.get("truth_inside_region")),
    }
    per_case = [{"case": r["case"], "label": r["label"], "n_bearings": r["n_bearings"],
                  "n_vertices": r.get("n_vertices"), "diameter_m": r.get("diameter_m"),
                  "circle_radius_m": r.get("circle_radius_m"),
                  "max_vertex_dist_m": r.get("max_vertex_dist_m"),
                  "ratio": r.get("ratio"), "excess_m": r.get("excess_m"),
                  "excess_pct": r.get("excess_pct"),
                  "coverage": r.get("diameter_circle_covers"),
                  "min_circle_over_half_diameter": r.get("min_circle_over_half_diameter"),
                  "worst_vertex_angle_deg": r.get("worst_vertex_angle_deg"),
                  "center_error_vs_truth_m": r.get("center_error_vs_truth_m"),
                  "truth_inside_region": r.get("truth_inside_region")} for r in results]
    meta = {"seed": seed, "n_cases": len(results),
            "coverage_tolerance_m": P.COVER_TOL,
            "halfplane_tolerance_m": P.DIST_TOL,
            "arena_radius_m": R_arena,
            "bearing_error_bound_deg": P.BEARING_ERR,
            "r_recv_range_m": [R_RECV_MIN, R_RECV_MAX],
            "case_files": [f"cases/" + os.path.basename(c) for c, _ in case_paths],
            "sweeps": {k: {p: s[p] for p in SWEEP_PARAM_KEYS if p in s}
                       for k, s in sweeps.items()}}
    return {"meta": meta, "cases_summary": cases_summary, "per_case": per_case,
            "sweeps": sweeps}

def empty_sweep(base, with_error):
    """--skip-sweep 时的占位结果：字段与真实扫描一致，仅数值为空。"""
    d = {"n_trials": base["n_trials"], "seed": base["seed"],
         "with_error": bool(with_error),
         "bearing_error_deg": P.BEARING_ERR if with_error else 0.0,
         "n_bounded": 0, "n_fail": 0, "fail_pct": None, "worst_ratio": None,
         "worst_config": None, "n_unbounded": 0, "n_empty": 0, "n_degenerate": 0}
    if "r_max" in base:
        d["r_max_m"] = base["r_max"]
    if "min_gap" in base:
        d["min_gap_deg"] = base["min_gap"]
        d["dist_range_m"] = list(base["dist"])
        d["m_choices"] = list(base["m_choices"])
        d["per_m"] = {}
    return d

def validate_cases(case_paths):
    """复算五组算例的裁剪误差、直径及真值约束，保存正文校验量。"""
    max_vertex_error = max_diameter_error = 0.0
    spans, gaps, distances = [], [], []
    for csv_path, truth_path in case_paths:
        with open(csv_path, encoding="utf-8") as f:
            rows = list(csv.DictReader(f))
        with open(truth_path, encoding="utf-8") as f:
            truth = json.load(f)
        pts = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
        svds = [float(r["svd_deg"]) for r in rows]
        V, status = P.region_from_bearings(pts, svds)
        C, _ = P.region_by_clipping(pts, svds)

```

```

        error = max(min(float(np.linalg.norm(v - c)) for c in C) for v in V)
        D, _ = P.diameter(V)
        Dc, _ = P.rotating_calipers(V)
        assert status == "bounded" and len(V) == len(C) and error < 1e-6
        assert abs(D - Dc) < 1e-9
        assert P.run_case_csv(csv_path, truth_path)["truth_inside_region"]
        max_vertex_error = max(max_vertex_error, error)
        max_diameter_error = max(max_diameter_error, abs(D - Dc))
        spans.append(truth["azimuth_span_deg"])
        gaps.append(truth["min_azimuth_gap_deg"])
        distances.extend(float(row["dist_m"]) for row in rows)
    return {"cases_checked": len(case_paths),
            "max_vertex_difference_m": max_vertex_error,
            "max_diameter_difference_m": max_diameter_error,
            "azimuth_span_deg_range": [min(spans), max(spans)],
            "min_azimuth_gap_deg": min(gaps),
            "source_distance_m_range": [min(distances), max(distances)]}

def main(argv=None):
    ap = argparse.ArgumentParser(description="问题一：算例生成 + 逐例求解 + 四组随机扫描")
    ap.add_argument("--out", default=os.path.dirname(os.path.abspath(__file__)),
                    help="算例与结果的输出目录（默认为本文件所在目录）")
    ap.add_argument("--seed", type=int, default=DEFAULT_SEED, help="算例生成种子")
    ap.add_argument("--cases", type=int, default=5, help="算例组数")
    ap.add_argument("--skip-sweep", action="store_true", help="跳过四组随机扫描")
    args = ap.parse_args(argv)
    if args.cases < 1:
        ap.error("算例数须为正整数")

    out = os.path.abspath(args.out)
    os.makedirs(out, exist_ok=True)
    case_dir = os.path.join(out, "cases")
    os.makedirs(case_dir, exist_ok=True)
    result_dir = os.path.join(out, "results")
    os.makedirs(result_dir, exist_ok=True)
    case_paths = gen_cases(case_dir, args.seed, args.cases)
    results = run_cases(case_paths)

    jl = os.path.join(result_dir, "p1_results.jsonl")
    with open(jl, "w", encoding="utf-8") as f:
        for r in results:
            f.write(json.dumps(r, ensure_ascii=False) + "\n")

    if args.skip_sweep:
        sweeps = {"two_point": empty_sweep(SWEEP_TWO_POINT, False),
                  "two_point_err": empty_sweep(SWEEP_TWO_POINT, True),
                  "well_conditioned": empty_sweep(SWEEP_WELL_COND, False),
                  "well_conditioned_err": empty_sweep(SWEEP_WELL_COND, True)}
    else:
        sweeps = {"two_point": sweep_two_point(),
                  "two_point_err": sweep_two_point(with_error=True),
                  "well_conditioned": sweep_well_conditioned(),
                  "well_conditioned_err": sweep_well_conditioned(with_error=True)}

    summary = build_summary(case_paths, results, sweeps, args.seed)
    summary["validation"] = validate_cases(case_paths)
    summary_name = "p1_cases_only.json" if args.skip_sweep else "p1_summary.json"
    with open(os.path.join(result_dir, summary_name), "w", encoding="utf-8") as f:
        json.dump(summary, f, ensure_ascii=False, indent=2)

    print("== 逐例结果 (%s) == % out)

```

```

for r in results:
    print("%-4s %-16s %-10s D=%-9s D/2=%-9s max|P-M|=%-9s ratio=%-9s cover=%-5s r*/D*2=%-8s" % (
        r["label"], r["case"], r["status"], r.get("diameter_m", "-"),
        r.get("circle_radius_m", "-"), r.get("max_vertex_dist_m", "-"),
        r.get("ratio", "-"), r.get("diameter_circle_covers", "-"),
        r.get("min_circle_over_half_diameter", "-")))
if not args.skip_sweep:
    print("\n== 随机扫描 (理想构型 / 实际构型: 同构型叠加  $\pm 0.0^\circ$  测向误差) =="
        % P.BEARING_ERR)
    for key, name in (("two_point", "两测点 理想"),
                      ("two_point_err", "两测点 实际"),
                      ("well_conditioned", "3-6 测点 理想"),
                      ("well_conditioned_err", "3-6 测点 实际")):
        s = sweeps[key]
        print("%-14s: trials=%d bounded=%d fail=%d (%.4f%%) worst_ratio=%s "
            " [unbounded=%d empty=%d degenerate=%d rejected=%d]" % (
                name, s["n_trials"], s["n_bounded"], s["n_fail"], s["fail_pct"],
                "-" if s["worst_ratio"] is None else "%.6f" % s["worst_ratio"],
                s["n_unbounded"], s["n_empty"], s["n_degenerate"],
                s["n_rejected_geometry"]))
print("\n== 论文表 2 行 ==")
for r in results:
    lab = r["A\textsubscript{s}"] % r["label"].lstrip("AC")
    print("
        %s & %d & %d & %.2f & %.2f & %.2f & %.4f & %s \\\\" % (
        lab, r["n_bearings"], r["n_vertices"], r["diameter_m"],
        r["circle_radius_m"], r["max_vertex_dist_m"], r["ratio"],
        "是" if r["diameter_circle_covers"] else "否"))
print("\n写出: %s, %s, cases/p1_case*.csv, cases/p1_case*.truth.json"
    % (os.path.join(result_dir, summary_name), jl))
return 0

if __name__ == "__main__":
    sys.exit(main())

```

2.3 问题一绘图

Listing 4: Q1/make_figures.py

```

"""由算例和几何求解器重绘问题一的三幅正文图。"""
from pathlib import Path
import csv
import os
import tempfile

os.environ.setdefault("MPLCONFIGDIR", str(Path(tempfile.gettempdir()) / "b-locator-mpl"))
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import Circle, FancyBboxPatch, Polygon
import numpy as np

import p1_intersection as P
import p1_experiments as E

HERE = Path(__file__).resolve().parent
PAPER = HERE.parent / "paper"
OUT = PAPER / "figures"
BLUE, RED, ORANGE = "#2463a5", "#b73432", "#cc6500"

```

```

def setup():
    font = PAPER / "fonts" / "simsun.ttc"
    if font.exists():
        font_manager.fontManager.addfont(font)
        plt.rcParams["font.family"] = font_manager.FontProperties(fname=font).get_name()
    plt.rcParams.update({"font.size": 11, "mathtext.fontset": "stix",
                        "axes.unicode_minus": False, "savefig.dpi": 300})
    OUT.mkdir(parents=True, exist_ok=True)

def save(fig, name):
    fig.savefig(OUT / name, bbox_inches="tight", facecolor="white")
    plt.close(fig)

def draw_region(ax, vertices, labels=True):
    D, (A, B) = P.diameter(vertices)
    M = (A + B) / 2
    ax.add_patch(Polygon(vertices, facecolor="#f2d0ce", edgecolor=RED, lw=1.5))
    ax.add_patch(Circle(M, D / 2, fill=False, color=BLUE, lw=1.5))
    ax.plot([A[0], B[0]], [A[1], B[1]], "k-", lw=1.2)
    ax.scatter(*np.array([A, B]).T, facecolor="white", edgecolor="black", zorder=5)
    ax.plot(*M, "k+", ms=9, mew=1.2)
    if labels:
        for name, point in (("A", A), ("B", B), ("M", M)):
            ax.annotate(f"${name}$", point, xytext=(5, 5), textcoords="offset points")
    ax.set_aspect("equal")
    ax.set_xlim(M[0] - 0.74 * D, M[0] + 0.74 * D)
    ax.set_ylim(M[1] - 0.74 * D, M[1] + 0.74 * D)
    return D, A, B, M

def coverage():
    fig, axes = plt.subplots(1, 2, figsize=(8.2, 4.2), constrained_layout=True)
    for i, ax in enumerate(axes, 1):
        with (HERE / "cases" / f"pi_case{i:02d}.csv").open(encoding="utf-8") as f:
            rows = list(csv.DictReader(f))
            points = [(float(r["x_m"]), float(r["y_m"])) for r in rows]
            bearings = [float(r["svd_deg"]) for r in rows]
            vertices, _ = P.region_from_bearings(points, bearings)
            D, A, B, M = draw_region(ax, vertices)
            full = E.solve_full(points, bearings)
            status = "覆盖成立" if full["covers"] else "覆盖不成立"
            ax.set_title(f"({'ab'[i-1]}) ${A_i}$: {status}\n"
                        f"$D={D:.2f}$ m, $\max |P-M|={full['max_vertex_dist']:.2f}$ m",
                        fontsize=11, color=BLUE if full["covers"] else ORANGE)
            if not full["covers"]:
                worst = vertices[np.argmax(np.linalg.norm(vertices - M, axis=1))]
                ax.plot([M[0], worst[0]], [M[1], worst[1]], "--", color=ORANGE)
                ax.scatter(*worst, s=55, facecolor="white", edgecolor=ORANGE, zorder=6)
                ax.annotate(f"$P$ ($\{full['worst_vertex_angle_deg']:.1f\}^\circ$)", worst,
                            xytext=(-15, -22), textcoords="offset points", color=ORANGE)
            ax.set_xlabel("$x$ / m")
            ax.set_ylabel("$y$ / m")
            ax.tick_params(labelsize=9)
        save(fig, "pi-coverage-contrast.png")

def flowchart():
    fig, ax = plt.subplots(figsize=(7.6, 4.4))

```

```

ax.set(xlim=(0, 10), ylim=(0, 6.8))
ax.axis("off")
steps = [(6.2, "输入检测点与示向度"),
          (5.05, "边界交点枚举、半平面筛选、凸包"),
          (3.9, "检查公共延伸方向"),
          (2.75, "检查顶点数"),
          (1.6, "枚举最远顶点对, 求  $D$  与  $M$ "),
          (0.45,  $r$  比较  $\max_{P \in V} |P-M|$  与  $D/2$ , 输出覆盖判定)]
for y, label in steps:
    ax.add_patch(FancyBboxPatch((0.6, y - 0.36), 6.4, 0.72,
                                boxstyle="round,pad=0.05", facecolor="#edf3f8", edgecolor=BLUE))
    ax.text(3.8, y, label, ha="center", va="center", fontsize=11)
for (y1, _), (y2, _) in zip(steps, steps[1:]):
    ax.annotate("", (3.8, y2 + 0.39), (3.8, y1 - 0.39),
                arrowprops={"arrowstyle": "→", "color": BLUE})
for y, label in [(5.05, "无交点: 空集"), (3.9, "存在: 无界"), (2.75, "少于 3: 退化")]:
    ax.annotate("", (7.9, y), (7.07, y), arrowprops={"arrowstyle": "→"})
    ax.text(8.05, y, label, ha="left", va="center", fontsize=10)
save(fig, "pi-algorithm-flow.png")

def wedge_diagram():
    T = np.array([0.0, 0.0])
    sensors = np.array([[-900., -250.], [700., -450.], [300., 850.]])
    bearings = [P.bearing(s, T) for s in sensors]
    vertices, _ = P.region_from_bearings(sensors, bearings)
    fig, ax = plt.subplots(figsize=(6.3, 5.7))
    ax.set(xlim=(-1200, 1200), ylim=(-900, 1100), xlabel=" $x$  / m", ylabel=" $y$  / m")
    ax.set_aspect("equal")
    for i, (s, theta, color) in enumerate(zip(sensors, bearings, [BLUE, ORANGE, "#49864e"]), 1):
        angles = np.deg2rad([theta - P.BEARING_ERR, theta + P.BEARING_ERR])
        ends = s + 2200 * np.column_stack([np.cos(angles), np.sin(angles)])
        ax.add_patch(Polygon(np.vstack([s, ends]), facecolor=color, alpha=0.15))
        for end in ends:
            ax.plot([s[0], end[0]], [s[1], end[1]], color=color, lw=0.8)
        ax.scatter(*s, color=color, s=30, zorder=4)
        ax.annotate(f" $S_{i}$ ", s, xytext=(6, 6), textcoords="offset points", color=color)
    ax.scatter(*T, color=RED, s=18, zorder=6)
    ax.annotate(f" $T$ ", T, xytext=(-12, 8), textcoords="offset points")
    ax.add_patch(Polygon(vertices, facecolor=RED, edgecolor=RED))
    inset = ax.inset_axes([0.57, 0.045, 0.40, 0.40])
    draw_region(inset, vertices, labels=False)
    for i, v in enumerate(vertices, 1):
        inset.annotate(f" $V_{i}$ ", v, xytext=(3, 4), textcoords="offset points", fontsize=8)
    _, (A, B) = P.diameter(vertices)
    inset.annotate(f" $M$ ", (A + B) / 2, xytext=(4, 4), textcoords="offset points", fontsize=9)
    inset.set_title("定位区域局部放大", fontsize=10)
    inset.tick_params(labelsize=7)
    ax.indicate_inset_zoom(inset, edgecolor="gray", alpha=0.65)
    ax.tick_params(labelsize=9)
    save(fig, "multi_sensor_wedge_diagram.png")

def main():
    setup()
    coverage()
    flowchart()
    wedge_diagram()
    print(f"问题一正文图已生成: {OUT}")

if __name__ == "__main__":

```

```
main()
```

附录 C 问题二主要源程序

问题二的完整源程序（支撑材料 Q2/src/q2_solution.py）见下，仅保留与正文式 (11)–(18) 直接对应的部分：交会四边形顶点、四边形直径与加权期望；寻优与等值区域统计略去，完整脚本见支撑材料。

Listing 5: 问题二：期望定位区域直径与最优点、候选区域（Python 3）

```
import math
import numpy as np

X_MAX, ALPHA_DEG = 1500.0, 1.0          # 有效接收半径；示向度误差界
ALPHA = math.radians(ALPHA_DEG)        # 探测束半张角

def quad_vertices(x_t, a, b, alpha=ALPHA):
    """S1 的两条边界射线（与 x 轴成 +-alpha）与 S2 的两条边界射线
    （绕 S2→S 方向成 +-alpha）两两求交，得交会四边形 ABCD 的四个顶点。"""
    s2 = np.array([a, b])
    psi = math.atan2(-b, x_t - a)      # S 相对 S2 的方位角
    pts = []
    for t1 in (alpha, -alpha):
        u1 = np.array([math.cos(t1), math.sin(t1)])
        for t2 in (psi + alpha, psi - alpha):
            u2 = np.array([math.cos(t2), math.sin(t2)])
            den = u1[0] * u2[1] - u1[1] * u2[0]
            if abs(den) < 1e-14:        # 两射线平行：定位区域无界
                return None
            s = (s2[0] * u2[1] - s2[1] * u2[0]) / den
            pts.append(s * u1)
    return np.array(pts)

def quad_diameter(x_t, a, b, alpha=ALPHA):
    """定位区域直径 D = 四边形顶点两两距离的最大值。"""
    p = quad_vertices(x_t, a, b, alpha)
    if p is None:
        return math.inf
    d = p[:, None, :] - p[None, :, :]
    return float(np.sqrt((d ** 2).sum(-1)).max())

def dbar(a, b, n=800, alpha=ALPHA):
    """期望定位区域直径：按  $P_T(x_T|S1) = 2 x_T / x_{max}^2$  加权求和。"""
    if abs(b) < 1e-9:                # 两点与源点共线，区域无界
        return math.inf
    x_t = X_MAX * (np.arange(n) + 0.5) / n
    w = 2.0 * x_t / X_MAX ** 2
    d = np.array([quad_diameter(float(x), a, b, alpha) for x in x_t])
    if not np.isfinite(d).all():
        return math.inf
    return float((w * d).sum() / w.sum())

if __name__ == "__main__":
    # 自检：粗网格上的最优相对位形
    best = min(((dbar(x * 1.0, y * 1.0), x, y)
                for x in range(950, 1251, 25) for y in range(500, 901, 25)),
```



```
key=lambda t: t[0])  
print("Dbar_min = %.2f m at (a,b) = (%.0f, %.0f) m" % best)
```

附录 D 问题三、四主程序（待补）

问题三、问题四的策略与算法完成后，其完整可运行源程序将补充于此（对应支撑材料 code/main.py）；该程序负责与模拟器交互，完成多源搜索、定位与清除的全过程。